

Exceptional Control Flow

Aufgabe 1

Um eine Interrupt Quelle nutzen zu können muss der entsprechende Interrupt enabled werden (neben anderen Konfigurationsaktionen).

Annahme: Sie müssen für den **Timer 2** einen Handler installieren und den Interrupt enablen.

- 1) Wie heisst der Interrupt Handler für **Timer 2**?
Öffnen Sie dazu in irgendeinem der Praktika Projekte das File `startup_ctboard.s` und suchen sie nach dem passenden Handler Namen in der `__Vectors` Tabelle.
Tipp: Suchen Sie nach `TIM2`...

`TIM2_IRQHandler`

- 2) Welche Interrupt Nummer hat dieser externe Interrupt? Zählen Sie vom ersten externen Interrupt, beginnend mit 0, bis zum entsprechenden Interrupt Handler.

`28`

- 3) Geben Sie die Assembler Instruktionen an um diesen Interrupt mit der gegebenen Interrupt Nummer zu enablen – siehe Vorlesung Seite 27: „Interrupt Enable Registers“.

```
.equ SETENA0, 0xE000E100
...
ldr  r7,=SETENA0
movs r6,#1
lsls r6,#28
str  r6,[r7]
```

Aufgabe 2

Enabling von Interrupt Quellen, wie in Aufgabe 1 behandelt, dient zur initialen Konfiguration von Interrupts. Im Betrieb ist es aber unter Umständen nötig alle Interrupts kurzfristig auszuschalten und danach wieder einzuschalten.

- 1) Was kann ein Grund sein für ein solches kurzfristiges Aus- und wieder Einschalten?

Data Konsistenz: z.B. eine Interrupt Service Routine eines Timers unterhält zwei Zähler,

einen für Minuten und einen für Stunden. Beim Übergang von Minute 59 zu 0 wird der

Stundenzähler um eins erhöht. Eine andere Routine welche diese beiden Zähler ausliest

muss dafür sorgen dass kein Interrupt passiert zwischen dem Zugriff auf diese beiden Zähler.

- 2) Wie schalten Sie alle Interrupts in **Assembler** aus? Wie ein?

Ausschalten:

```
CPSID    i
```

Einschalten:

```
CPSIE    i
```

- 3) Wie schalten Sie alle Interrupts in C aus? Wie ein?

Ausschalten:

```
__disable_irq();
```

Einschalten:

```
__enable_irq();
```

Aufgabe 3

Der ARM Prozessor rettet beim Abarbeiten eines Interrupts gewisse Register auf den Stack bevor die ISR (Interrupt Service Routine) ausgeführt wird – und restauriert diese nach Beendigung der ISR automatisch.

- 1) Wenn Sie in Ihrer ISR die Register R0-R6 verwenden, welche dieser Register müssen Sie auf den Stack pushen weil sie nicht schon automatisch vorher gerettet wurden?

`push {r4-r6} @ r0-r3 wurden schon automatisch gerettet`

- 2) Wie unterscheidet sich in der Programmierung eine ISR von einer „normalen“ Funktion?

Jede ISR hat einen vordefinierten Namen (von der `__Vectors` Tabelle vorgegeben).

Eine ISR kann keine Werte zurückgeben (ist in C eine `void isr_name(void)` Funktion).

Eine ISR muss gegebenenfalls den Interrupt zurücksetzen so dass er nicht permanent feuert.
